# Activation Placement and Memory Contention in Microcontroller-Class Neural Inference

Alp O. Bolukbasi
*Department of Informatics*
*Karlsruhe Institute of Technology*
Karlsruhe, Germany

***Abstract*—Moving a neural network's activation buffer between on-chip SRAM regions changes inference time on an STM32U585 by three cycles out of 320,898 when no other bus master is active. With a DMA engine active, the same placement choice changes the cost by up to 31,189 cycles. Both results come from one firmware image and a cycle-accurate telemetry path whose record insertion cost is 118 cycles outside the measured interval. The uncontended effect is 9.3 parts per million. The shift is consistent at the minimum and median, while the upper tails overlap, so it is measurable but too small to serve as a scheduling margin. Under contention the effect reaches 9.7 percent and is asymmetric. Traffic in SRAM3 adds about 4.7 percent to arenas in SRAM1 and SRAM2 without sharing a region with either, while traffic in SRAM1 or SRAM2 leaves an arena in SRAM3 close to baseline at the reported working set. These measurements support treating activation placement as a scheduling variable only when the memory system is shared. They also leave the SRAM3 asymmetry as the part of the placement cost model that still requires explanation.**

***Index Terms*—embedded machine learning, memory interference, scratchpad allocation, real-time systems, microcontrollers**

## I. Introduction

Activation placement changes inference time by three cycles out of 320,898 on an idle STM32U585. The same choice changes the cost by up to 31,189 cycles when a second bus master is active. The first effect is measurable but operationally negligible. The second is large enough to affect a placement decision.

Neural network inference on microcontrollers is usually treated as a compute problem. The activation arena, the scratch buffer that stores intermediate tensors during inference, is then treated as a size constraint. It either fits in on-chip SRAM or the network cannot run in that configuration. Its address is normally left to the linker.

That default is weaker on a microcontroller where DMA engines, sensor peripherals, and network drivers can move data while the core runs inference. Those requesters do not consume the inference thread's scheduled CPU time. They can still share the memory interconnect with it. If activation placement changes that interference cost, placement becomes a run-time decision rather than only a linker detail.

This work measures that effect and does not implement a placement policy. A later policy would only be useful if the placement cost is measurable, stable enough to use, and large enough to change a scheduling decision. The measurement therefore has to resolve a near-null result and a positive result with the same instrumentation.

Without a competing master, the placement spread is three cycles, or 9.3 parts per million. With GPDMA traffic, same-region contention reaches 9.7 percent. Cross-region traffic exposes a directional SRAM3 effect that is not explained by region size or by a simple same-region versus different-region model.

## II. Background and Related Work

Scratchpad allocation is a mature embedded-systems problem. Banakar et al. made the case for software-managed on-chip memory as an alternative to caches in embedded designs [2]. Avissar et al. [3] and Steinke et al. [4] formulated object placement in scratchpad memory using static cost models. Those models optimize where objects live. They do not ask how that cost changes when an independent master is active at the same time.

Memory interference has also been studied on multicore systems with shared DRAM. MemGuard regulates per-core memory bandwidth to recover temporal isolation [5]. PALLOC separates allocations across DRAM banks so placement itself reduces interference [6]. Kim et al. derive bounds for memory interference delay [7]. Pellizzoni et al. separate execution into phases that avoid uncontrolled contention [8]. Those systems have caches, DRAM controllers, multiple cores, or some combination of them. The STM32U585 exposes uncached internal SRAM to one Cortex-M33 core and several independent bus masters. The interference mechanism has to be measured again.

TinyML work normally treats the activation arena as a capacity problem. TensorFlow Lite Micro uses the arena as its main memory abstraction for microcontroller inference [9]. MLPerf Tiny defines reproducible benchmarks for this deployment class [10]. MCUNet and MCUNetV2 reduce peak activation demand so larger models fit on small devices [11], [12]. Those works ask how much memory the network needs. This work asks whether the address of that memory changes execution time once another master shares the interconnect.

The name Laxity follows the real-time use of laxity as the slack between remaining execution and a deadline [1]. In this experiment, placement only matters if it changes slack by enough cycles for a later scheduler to spend.

## III. System Design

Figure 1 summarizes the measurement architecture implemented in the Laxity repository [15]. It separates experiment tooling and host analysis from the embedded target. Figure 2 isolates the experimental variable: the activation arena and DMA working set are selected independently among the three tested SRAM regions. The second figure is an experiment diagram, not a claim about the undocumented internal bus topology. SRAM capacities and addresses follow RM0456 [14].

### A. Platform

Measurements were taken on a B-U585I-IOT02A board carrying an STM32U585. The Cortex-M33 runs at 160 MHz with four flash wait states and the regulator in its highest voltage scaling mode. Instructions are fetched from flash with the instruction cache enabled. The data cache covers only the external memory aperture from `0x60000000` to `0x9FFFFFFF`, so internal SRAM accesses are not absorbed by the data cache. The activation arena therefore remains visible to the on-chip memory system during inference.

The device provides 192 KiB of SRAM1 at `0x20000000`, 64 KiB of SRAM2 at `0x20030000`, and 512 KiB of SRAM3 at `0x20040000` [14]. A fourth SRAM in the low-power domain was excluded because its intended use is not comparable to the three regions under test. The reference manual documents CPU and DMA access to the tested regions through the device memory system [14].

Application code runs under ThreadX. Inference uses a network converted by ST Edge AI Core 4.0.1 from a public human activity recognition model trained on the WISDM accelerometer dataset [13]. The network accepts 24 samples across three axes and returns four class scores. Its activation arena is 2,944 bytes. That size is small relative to every tested SRAM region and is a limit of the study.

### B. Instrumentation

Inference is timed with the Data Watchpoint and Trace cycle counter. The counter is 32 bits wide and wraps after 26.8 seconds at 160 MHz, so wrap is detected and recorded rather than assumed absent. Each completed inference produces a fixed 32-byte record with a sequence number, release timestamp, execution interval, arena identity, and competing-master identity. Records enter a single-producer, single-consumer ring and leave the target in framed batches protected by a CRC.

The instrumentation was measured before using it to measure inference. A null probe exercises the record path without running the network. Reading the cycle counter takes 14 cycles at both the median and p99. Pushing one record takes 118 cycles at both statistics over 128 probes. The push occurs outside the measured inference interval. It is 0.037 percent of one 320,898-cycle inference.

Sequence numbers advance even when a record is dropped. Otherwise a full telemetry ring could lose a sample without leaving evidence on the host. A gap in the received sequence is therefore the observable sign of a dropped record. The reported capture has no such gaps.

### C. Placement Mechanism

The firmware contains one statically allocated activation buffer in each tested SRAM region. The buffers have identical alignment. A pointer selects the active buffer at run time, so changing the active region does not require a rebuild or relink. The converted network receives its activation storage through its run-time interface. Inputs and outputs are allocated inside that activation storage, so selecting another arena moves the tensor storage with it.

Run-time selection is required for this measurement. A relink of identical sources moved the observed median by 85 cycles in earlier measurements on the same system. The uncontended placement spread reported here is only three cycles. A build-per-placement experiment would mix placement with image layout. Every result in Section V comes from one firmware image.

The capture identifies that image using a hash of the flashed binary rather than only the ELF container. A clean rebuild can reorder debug sections while leaving the loadable image unchanged. An ELF hash would then reject two binaries that execute identically. The image hash keeps that distinction outside the measurement.

### D. Competing Master

The competing master is a GPDMA1 channel performing memory-to-memory transfers through a circular linked list. Source and destination both occupy the SRAM region selected for that cell. Source and destination transfers use different GPDMA matrix ports. The channel runs at high priority with single-word bursts. No DMA interrupt is enabled. Completion progress is polled outside the inference interval so DMA accounting cannot add CPU execution to the measured window.

Using DMA instead of a second thread is the central methodological choice. The STM32U585 has one core and no simultaneous multithreading. Two software threads cannot execute at the same instant. A thread that preempts inference contributes its own execution time to the measured interval. Increasing such a thread's activity would measure scheduler accounting together with any memory effect. GPDMA is independent of the core. It gives the memory system two simultaneous requesters without placing aggressor instructions inside the timed CPU interval.

## IV. Methodology

The main measurement matrix crosses three arena regions with three aggressor regions and one aggressor-off condition. That gives twelve cells. Two controls test whether a small between-region difference could come from the measurement path or from the chosen address. The SRAM1 control assigns a second identity to the same buffer, so its median spread provides a same-address noise floor. A second address in SRAM2 changes the address while keeping the SRAM region fixed.

Cells are visited in randomized order across repetitions, so drift does not accumulate against one placement. The system is warmed before recording. The null probe runs before each

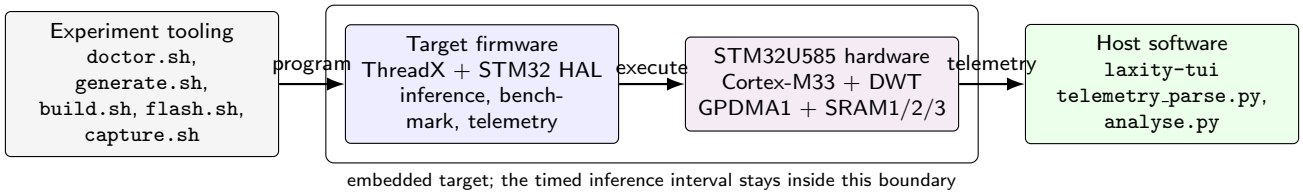


Fig. 1. Measurement architecture derived from the repository architecture diagram [15]. Experiment scripts program and capture the target. The target firmware runs under ThreadX, invokes ST Edge AI inference, drives the GPDMA aggressor, and records telemetry. Host software consumes the resulting stream. The timed inference interval remains on the embedded target.

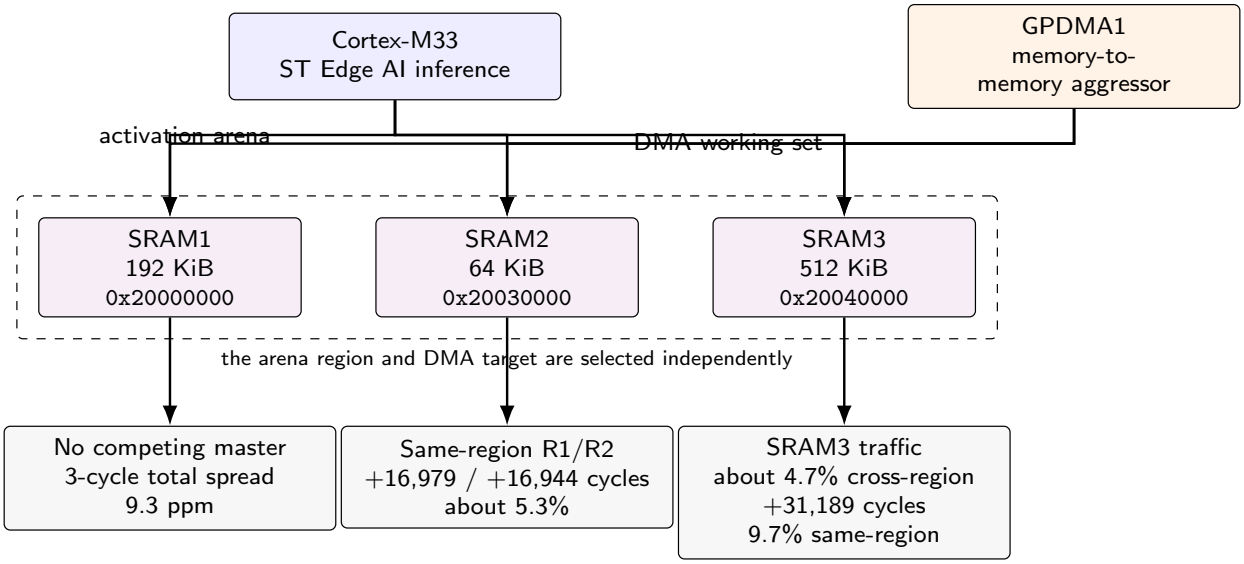


Fig. 2. Memory-contention experiment derived from the repository contention diagram [15]. The 2,944-byte activation arena and the DMA working set are selected independently across SRAM1, SRAM2, and SRAM3. The diagram summarizes measured outcomes and does not infer an undocumented internal bus topology.

session. Medians and 99th percentiles are reported because the observed distributions are narrow and multimodal. Reporting only the mean would hide both properties.

The reported capture contains 9,062 records over 180 seconds, with between 217 and 225 samples per cell. It contains no dropped records, no sequence gaps, and no CRC rejections. One cycle-counter wrap was detected and flagged. Raw captures store the build configuration, clock and voltage settings, and flashed image hash with the samples. The analysis script refuses to produce a result when required provenance is missing.

TABLE I
UNCONTENDED DISTRIBUTION SUMMARY FOR THE THREE PRIMARY REGIONS AND TWO CONTROLS. VALUES ARE INFERENCE CYCLES.

| Cell | $n$ | min | p50 | p99 | max |
|---|---|---|---|---|---|
| SRAM1 | 224 | 320,893 | 320,898 | 320,916 | 320,919 |
| SRAM2 | 222 | 320,896 | 320,901 | 320,914 | 320,917 |
| SRAM3 | 219 | 320,894 | 320,899 | 320,915 | 320,918 |
| SRAM1 ctrl | 220 | 320,893 | 320,898 | 320,911 | 320,924 |
| SRAM2 ctrl | 221 | 320,896 | 320,901 | 320,917 | 320,919 |

## V. RESULTS

### A. Placement without Contention

The first column of Table II gives the uncontended medians. SRAM1, SRAM2, and SRAM3 return 320,898, 320,901, and 320,899 cycles. The full range is three cycles out of 320,898, or 9.3 parts per million.

The same-address SRAM1 control returns the same 320,898-cycle median. The second address in SRAM2 returns the same 320,901-cycle median as the primary SRAM2 address. These controls show that the median estimate is stable at this sample size. They do not show that the full distributions are separated.

The uncontended distributions overlap in the upper tail. The three-cycle shift is visible at the minimum and the median, while each cell's p50-to-p99 span is 13 to 18 cycles. The result therefore describes a repeatable shift near the distribution floor, not separation of the full distributions and not a scheduling margin. The instrumentation resolves the shift, but its magnitude is operationally negligible.

TABLE II
MEDIAN INFERENCE TIME IN CYCLES BY ARENA REGION AND COMPETING-MASTER REGION. PARENTHESIZED VALUES ARE CHANGES FROM THE SAME ARENA'S UNCONTENDED MEDIAN.

| Arena | None | DMA in R1 | DMA in R2 | DMA in R3 |
|---|---|---|---|---|
| R1 (192 KiB) | 320,898 | 337,877 (+16,979) | 320,927 (+29) | 335,843 (+14,945) |
| R2 (64 KiB) | 320,901 | 320,960 (+59) | 337,845 (+16,944) | 335,847 (+14,946) |
| R3 (512 KiB) | 320,899 | 320,967 (+68) | 320,937 (+38) | 352,088 (+31,189) |

### B. Placement under Contention

Contention changes the scale from single cycles to tens of thousands. Table II shows the main capture. Values in parentheses are relative to the same arena's uncontended median.

When inference and DMA share SRAM1, the median increases by 16,979 cycles. SRAM2 gives almost the same result at 16,944 cycles. Both penalties are about 5.3 percent.

SRAM1 and SRAM2 barely affect each other when the arena and DMA buffer are separated. SRAM2 traffic adds 29 cycles to an SRAM1 arena. SRAM1 traffic adds 59 cycles to an SRAM2 arena. Those changes are around 0.01 percent of the uncontended inference time. For this workload, SRAM1 and SRAM2 behave as independent targets at the scale relevant to placement.

SRAM3 does not follow that pattern. An SRAM3 aggressor adds 14,945 cycles to an arena in SRAM1 and 14,946 cycles to an arena in SRAM2, about 4.7 percent in either case. The reverse pairings remain close to baseline. SRAM1 traffic adds 68 cycles to an SRAM3 arena, and SRAM2 traffic adds 38 cycles. Same-region interference is also larger in SRAM3. Running both arena and aggressor there adds 31,189 cycles, or 9.7 percent.

The experiment therefore gives two placement regimes. With an idle memory system, the full placement range is three cycles. With another bus master active, the range reaches 31,189 cycles. A placement policy should model current memory use rather than choose one SRAM region globally.

### C. Working Set of the Competing Master

The aggressor working-set sweep shows that contention cost is not fixed once a region is chosen. Across the non-flat cells, moving from a 1 KiB to a 16 KiB aggressor buffer changes the median by 423 to 568 cycles. The direction depends on the aggressor region.

Traffic in R1 or R2 becomes slightly less expensive as the working set grows. Traffic in R3 becomes slightly more expensive against arenas in R1 and R2. When both arena and aggressor occupy R3, the response changes by only 91 cycles across the same range.

Aggregating the sweep across cells produces a small downward trend because it combines effects with opposite signs. That aggregate does not describe either regional behavior. The sweep is therefore reported per cell.

TABLE III
MEDIAN INFERENCE TIME IN CYCLES AGAINST THE COMPETING MASTER'S WORKING-SET SIZE FOR SELECTED CELLS.

| Arena | Master | 1 KiB | 4 KiB | 8 KiB | 16 KiB |
|---|---|---|---|---|---|
| R1 | R1 | 338,414 | 337,981 | 337,898 | 337,877 |
| R1 | R3 | 335,420 | 335,746 | 335,816 | 335,843 |
| R2 | R2 | 338,391 | 337,951 | 337,869 | 337,845 |
| R2 | R3 | 335,421 | 335,747 | 335,819 | 335,847 |
| R3 | R3 | 351,997 | 352,052 | 352,066 | 352,088 |

### D. A Real Workload

The synthetic GPDMA master saturates the path it occupies. It is a ceiling, not a normal operating point. The firmware also runs a sensing pipeline to place the experiment beside realistic traffic. An inertial sensor sampled at 26 Hz feeds the network input window. An environmental sensor runs alongside it. A microphone is captured through the digital filter peripheral at a measured 16,230 samples per second against 16,233 samples per second predicted by its clock chain.

The audio path writes about 32 KiB per second, three orders of magnitude less traffic than the synthetic master. Its buffer resides in SRAM3, which is the region that produces the cross-region asymmetry in the synthetic experiment. That makes the sensing pipeline a useful operating case. It does not make the synthetic result a prediction of the sensing cost.

The pipeline is always enabled in the reported firmware and cannot currently be disabled at run time. Its timing contribution appears as a shift in the uncontended baseline together with the inertial and environmental paths. No separate number is assigned to the microphone path because the capture does not separate those contributions.

## VI. DISCUSSION AND LIMITATIONS

Table II shows that placement has negligible timing value when the memory system is idle. Three cycles out of 320,898 are measurable, but too small to justify run-time placement machinery at this scale. The same placement variable becomes material once GPDMA traffic is present.

SRAM3 is the unresolved part of the result. RM0456 documents a multilayer bus matrix with round-robin arbitration between masters and a fast path that connects each master to a selected slave without added latency [14]. Other slave paths for that master incur at least one additional cycle on a new access. The same manual allows both CPU and DMA engines to address SRAM1, SRAM2, and SRAM3 [14].

Those documented properties narrow the explanation. They do not select one. A fixed non-fast-path cost does not explain why traffic in SRAM3 penalizes arenas in both SRAM1 and SRAM2 while traffic in SRAM1 and SRAM2 leaves an arena in SRAM3 close to baseline. The transfer counter recorded with each sample also advances at the same rate for all three aggressor regions. The measurements therefore establish the asymmetry, but they do not identify its internal arbitration point.

Several limits remain attached to the result. The arena size is fixed at 2,944 bytes because it belongs to the converted network. The experiment does not show how interference scales with activation footprint. Measurements were taken at optimization level zero, which lengthens inference and can dilute any fixed-cost interference component. Each main region contributes one arena address, so region and address are otherwise confounded. The second address in SRAM2 reduces that concern for the three-cycle uncontended effect, but it does not test every address in every region.

The application workload adds two limits. The inertial sensor runs at 26 Hz while the dataset used to train the network was sampled at 20 Hz. That mismatch affects classification quality rather than the timing measured here. The competing master used one GPDMA channel during most development and a different channel after the audio driver claimed the first. Equal-priority arbitration is assumed not to depend on channel index, so channel identity is not treated as an experimental variable.

The network stack needs separate treatment. With the radio driver's threads running, inference costs about four percent more. Those threads have higher priority than the inference thread and can preempt it, so the observed increase mixes CPU preemption with any bus interference. A separate experiment would be needed before attributing that four percent to the memory system.

## VII. Conclusion and Future Work

Activation placement changes median inference time by three cycles out of 320,898 when no competing bus master is active. Under GPDMA contention, the measured cost reaches 31,189 cycles, or 9.7 percent. The same-address control has zero median spread, and the telemetry record path costs 118 cycles outside the measured inference interval. Those controls let the three-cycle result and the 31,189-cycle result come from the same measurement path without treating either as an instrumentation artifact.

The remaining explanatory problem is SRAM3. Traffic there adds about 4.7 percent to arenas in SRAM1 and SRAM2 even though the regions differ, while the reverse pairings remain close to their uncontended baselines. RM0456 describes the bus arbitration rules, but not enough of the internal path to assign that asymmetry to one documented mechanism. A placement cost model should retain the measured per-region relationship rather than replace it with an assumed topology.

A run-time placement policy is left as future work because the current data supports the cost model, not the policy. Such a policy would need to observe which memory regions are busy and compare the measured cost of the available placements before moving or admitting an inference job. The existing telemetry already carries the arena and aggressor identifiers needed to evaluate that decision. The missing part is the policy that consumes those measurements and a mechanism for validating its decisions under changing workloads.

## References


[1] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973.

[2] R. Banakar, S. Steinke, B.-S. Lee, M. Balakrishnan, and P. Marwedel, "Scratchpad memory: a design alternative for cache on-chip memory in embedded systems," in *Proc. CODES*, 2002, pp. 73–78.

[3] O. Avissar, R. Barua, and D. Stewart, "An optimal memory allocation scheme for scratch-pad-based embedded systems," *ACM Transactions on Embedded Computing Systems*, vol. 1, no. 1, pp. 6–26, 2002.

[4] S. Steinke, L. Wehmeyer, B.-S. Lee, and P. Marwedel, "Assigning program and data objects to scratchpad for energy reduction," in *Proc. DATE*, 2002, pp. 409–415.

[5] H. Yun, G. Yao, R. Pellizzoni, M. Caccamo, and L. Sha, "MemGuard: memory bandwidth reservation system for efficient performance isolation in multi-core platforms," in *Proc. RTAS*, 2013, pp. 55–64.

[6] H. Yun, R. Mancuso, Z.-P. Wu, and R. Pellizzoni, "PALLOC: DRAM bank-aware memory allocator for performance isolation on multicore platforms," in *Proc. RTAS*, 2014, pp. 155–166.

[7] H. Kim, D. de Niz, B. Andersson, M. Klein, O. Mutlu, and R. Rajkumar, "Bounding memory interference delay in COTS-based multi-core systems," in *Proc. RTAS*, 2014, pp. 145–154.

[8] R. Pellizzoni, E. Betti, S. Bak, G. Yao, J. Criswell, M. Caccamo, and R. Kegley, "A predictable execution model for COTS-based embedded systems," in *Proc. RTAS*, 2011, pp. 269–279.

[9] R. David *et al.*, "TensorFlow Lite Micro: embedded machine learning for TinyML systems," *Proc. Machine Learning and Systems*, vol. 3, pp. 800–811, 2021.

[10] C. Banbury *et al.*, "MLPerf Tiny benchmark," in *Proc. NeurIPS Datasets and Benchmarks Track*, 2021.

[11] J. Lin, W.-M. Chen, Y. Lin, J. Cohn, C. Gan, and S. Han, "MCUNet: tiny deep learning on IoT devices," in *Proc. NeurIPS*, 2020, pp. 11711–11722.

[12] J. Lin, W.-M. Chen, H. Cai, C. Gan, and S. Han, "MCUNetV2: memory-efficient patch-based inference for tiny deep learning," in *Proc. NeurIPS*, 2021, pp. 2346–2358.

[13] J. R. Kwapisz, G. M. Weiss, and S. A. Moore, "Activity recognition using cell phone accelerometers," *ACM SIGKDD Explorations Newsletter*, vol. 12, no. 2, pp. 74–82, 2010.

[14] STMicroelectronics, "STM32U5 series Arm-based 32-bit MCUs reference manual RM0456," Rev. 7, 2026.

[15] A. O. Bolukbasi, "Laxity: real-time inference middleware and memory-contention experiments for STM32U585," software repository, 2026. [Online]. Available: https://github.com/alpblkba/laxity